```
import os

filename = '/content/measurements.csv'

x = []
y = []

if os.path.exists(filename):
    with open(filename, 'r') as f:
        next(f)
        for line in f:
            parts = line.strip().split(',')
            if len(parts) == 2:
                x.append(float(parts[0]))
                y.append(float(parts[1]))
    print(f"Successfully loaded {len(x)} data points from {filename}.")
    print(f"First 5 x values: {x[:5]}")
    print(f"First 5 y values: {y[:5]}")
else:
    print(f"Error: The file '{filename}' was not found.")
```

```
Successfully loaded 41 data points from /content/measurements.csv.
First 5 x values: [0.0, 0.1, 0.2, 0.3, 0.4]
First 5 y values: [11.082729112423378, 10.655909249764516, 11.08237582967664, 10.873200236081404, 12.793836854131584]
```

```
from simple_linear_regression import least_squares_fit

alpha_measurements, beta_measurements = least_squares_fit(x, y)

print(f"Calculated Y-intercept (alpha) for measurements data: {alpha_measurements}")
print(f"Calculated slope (beta) for measurements data: {beta_measurements}")
```

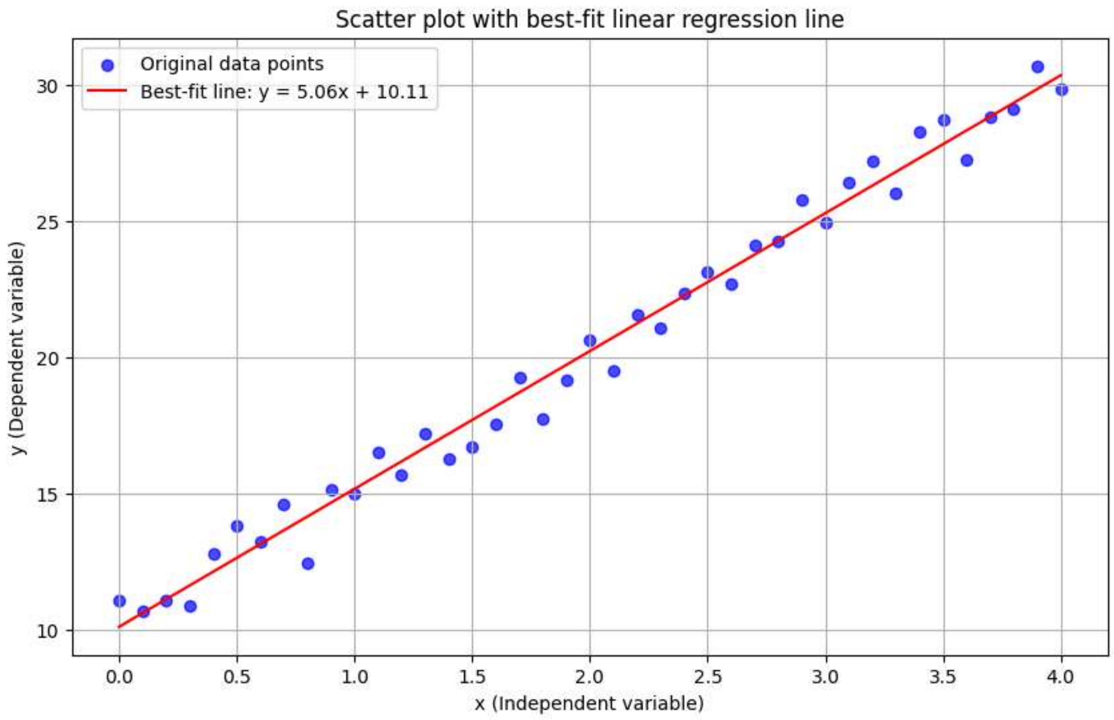
```
Calculated Y-intercept (alpha) for measurements data: 10.10517506089182
Calculated slope (beta) for measurements data: 5.063174354613681
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.scatter(x, y, label='Original data points', color='blue', alpha=0.7)

regression_line_y = [beta_measurements * val + alpha_measurements for val in x]
plt.plot(x, regression_line_y, color='red', linestyle='-', label=f'Best-fit line: y = {beta_measurements:.2f}x + {alpha_measurements:.2f}')

plt.xlabel('x (Independent variable)')
plt.ylabel('y (Dependent variable)')
plt.title('Scatter plot with best-fit linear regression line')
plt.grid(True)
plt.legend()
plt.show()
```




```
from simple_linear_regression import sum_of_sqerrors, r_squared

sse_measurements = sum_of_sqerrors(alpha_measurements, beta_measurements, x, y)
print(f"Sum of squared errors (SSE) for measurements data: {sse_measurements}")

r_squared_measurements = r_squared(alpha_measurements, beta_measurements, x, y)
print(f"R-squared (R2) for measurements data: {r_squared_measurements}")
```

```
Sum of squared errors (SSE) for measurements data: 23.25849000377901
R-squared (R2) for measurements data: 0.98443987596273
```

```
"""SSE: The calculated SSE for the measurements data is `23.26`. SSE measures the total squared difference between the observed values and the valu

R-squared (R2): The R-squared value for the measurements data is `0.9844`. R-squared represents the proportion of the variance in the dependent var

Overall quality: Both metrics indicate an excellent fit of the linear regression model to the csv data. The low SSE and very high R-squared value s
```

```
""" α (Y-intercept): In this context, α (approximately 10.11) represents the initial position of the object at time x = 0. It is the position where
β (Slope): β (approximately 5.06) represents the velocity of the object. It indicates the rate of change of position (y) with respect to time (x).
```

```
' α (Y-intercept): In this context, α (approximately 10.11) represents the initial position of the object at time x = 0. It is the position where the experim
ent began or the reference point from which time started being measured.\nβ (Slope): β (approximately 5.06) represents the velocity of the object. It indicat
es the rate of change of position (y) with respect to time (x). In physical terms, it tells us how many units of position the object moves per unit of time
(e.g., meters per second, if y is in meters and x in seconds).'
```

```
""" Q2 results:
SSE = 0.0
R-squared = 1.0

Q3 results:
SSE ≈ 23.26
R-squared ≈ 0.9844
The results are different because the nature of the data in the two scenarios is fundamentally different:

In Q2 (is deterministic data):
The y values were generated using a precise mathematical formula (y = 3x - 5) without any added random noise or error. When the least_squares_fit m
Why SSE = 0: Since the model's predictions exactly matched the actual y values for every data point, the errors (prediction - actual) were all zero
Why R-squared = 1: R-squared measures the proportion of variance in y explained by x. With SSE = 0, R-squared becomes 1 - (0 / TSS) = 1. This means

In Q3 (measurements data):
The measurements.csv data likely represents real-world observations, which inherently contain some level of random error, measurement inaccuracies,
Why SSE ≈ 23.26 (non-zero): The regression model, while providing a very good fit, cannot perfectly predict every y value because of the inherent r
Why R-squared ≈ 0.9844 (less than 1): While very high, R-squared is not 1 because the linear model does not explain 100% of the variance in y. The

So which one is more realistic? question/scenario 3, because real-world data, especially from experiments or observations, almost always contains s
```

```
' Q2 results:\nSSE = 0.0\nR-squared = 1.0\n\nQ3 results:\nSSE ≈ 23.26\nR-squared ≈ 0.9844\nThe results are different because the nature of the data in the tw
o scenarios is fundamentally different:\n\nIn Q2 (is deterministic data):\nThe y values were generated using a precise mathematical formula (y = 3x - 5) with
out any added random noise or error. When the least_squares_fit method was applied, it perfectly recovered the original parameters (α = -5 and β = 3).\nWhy S
SE = 0: Since the model's predictions exactly matched the actual y values for every data point, the errors (prediction - actual) were all zero. The sum of sq
uared zeros is zero, indicating a perfect fit.\nWhy R-squared = 1: R-squared measures the proportion of variance in y explained by x. With SSE = 0, R-squared
becomes 1 - (0 / TSS) = 1. This means 100% of the variability in y was perfectly explained by x through the linear model.\n\nIn Q3 (measurements data):\nThe
measurements.csv data likely represents real-world observa...'
```